

1.

```
knsat@SRITHIKA:~/LAB$ ./hello
Welcome to operating system
knsat@SRITHIKA:~/LAB$ |
```

```
GNU nano 0.7.1
#include<stdio.h>
int main(){
printf("Welcome to operating system\n");
return 0;
}
```

2.

```
knsat@SRITHIKA:~$ nano readusername.c
knsat@SRITHIKA:~$ gcc readusername.c -o readusername
knsat@SRITHIKA:~$ ./readusername
Enter your name: K.Srithika
Welcome K.Srithika
knsat@SRITHIKA:~$ |
```

```
GNU nano 0.7.1
#include <stdio.h>
int main() {
char name[30];
printf("Enter your name: ");
scanf("%s", name);
printf("Welcome %s\n", name);
return 0;
}
```

3.

```
knsat@SRITHIKA:~$ gcc display.c -o display
knsat@SRITHIKA:~$ ./display
Process ID = 994
knsat@SRITHIKA:~$ |
```

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Process ID = %d\n", getpid());
    return 0;
}
```

4.

```
#inknsat@SRITHIKA:~$ nano fork.c
#inknsat@SRITHIKA:~$ gcc fork.c -o fork
inknsat@SRITHIKA:~$ ./fork
Hello
Hello
}   knsat@SRITHIKA:~$ |
```

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    printf("Hello\n");
    return 0;
}
```

5.

```
knsat@SRITHIKA:~$ nano forkcalling.c
knsat@SRITHIKA:~$ gcc forkcalling.c -o forkcalling
knsat@SRITHIKA:~$ ./forkcalling
Hello
Hello
Hello
Hello
knsat@SRITHIKA:~$ |
```

```

#include <stdio.h>
#include <unistd.h>
int main() {
    fork();    // First fork
    fork();    // Second fork
    printf("Hello\n");
    return 0;
}

```

6.

```

knsat@SRITHIKA:~$ nano hello.c
knsat@SRITHIKA:~$ gcc hello.c -o hel
knsat@SRITHIKA:~$ ./hello
i am Parent Process
i am Child Process
knsat@SRITHIKA:~$ |

```

```

#include<stdio.h>
#include<unistd.h>
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        printf("i am Child Process\n");
    }
    else
    {
        printf("i am Parent Process\n");
    }
    return 0;
}

```

7.

```

knsat@SRITHIKA:~$ ./wait
Child Running
Parent Running After Child
knsat@SRITHIKA:~$ |

```

GNU nano 8.7.1

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Running After Child\n");
    }
    return 0;
}
```